

## **CSA0910 – JAVA PROGRAMMING**

### **SCENARIO BASED ASSIGNMENT**

**NAME : S.ROHINI**

**REG.NO : 192424413**

**DATE : 17/07/2026**

#### **1. Single Inheritance – Scenario**

##### **Scenario:**

A company has a Vehicle class with methods startEngine() and stopEngine(). A Car class extends Vehicle and adds a playMusic() method.

The company now wants every car to automatically log the engine start time whenever startEngine() is called, while still using the original engine-start functionality.

##### **Questions:**

1. How would you implement this using single inheritance?

```
import java.time.LocalDateTime;
```

```
class Vehicle {
```

```
    void startEngine() {
```

```
        System.out.println("Engine started");
```

```
    }
```

```
    void stopEngine() {
```

```
        System.out.println("Engine stopped");
```

```
    }
```

```
}
```

```
class Car extends Vehicle {
```

```
    @Override
```

```
    void startEngine() {
```

```

        super.startEngine(); // Calls parent class method

        System.out.println("Engine start time logged: " + LocalTime.now());
    }

    void playMusic() {

        System.out.println("Playing music");

    }

}

public class Main {

    public static void main(String[] args) {

        Car car = new Car();

        car.startEngine();

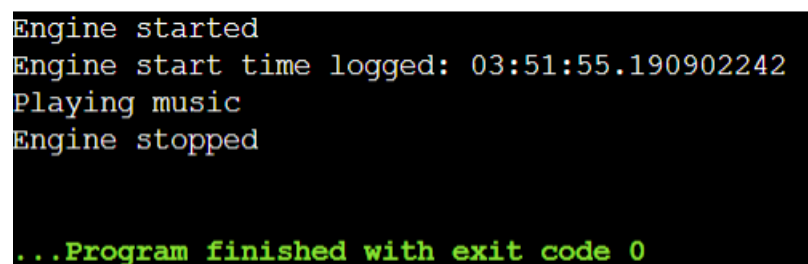
        car.playMusic();

        car.stopEngine();

    }

}

```



```

Engine started
Engine start time logged: 03:51:55.190902242
Playing music
Engine stopped

...Program finished with exit code 0

```

2. Should Car override startEngine()? Why?

Yes. Overriding is the only way to add the logging behavior while keeping the method name and calling convention (car.startEngine()) unchanged for all callers.

3. How can the parent class implementation still be executed?

By calling super.startEngine() inside the overridden method — this invokes Vehicle's version before (or after) the added logging code runs.

4. What happens if Car does not override startEngine()?

Calling startEngine() on a Car object simply runs Vehicle's original method (inherited as-is), with no logging — the new requirement wouldn't be met.

---

## 2. Multilevel Inheritance – Scenario

### Scenario:

A software company models employees as follows:

- Employee → stores employee ID and name.
- Developer extends Employee → stores programming language.
- SeniorDeveloper extends Developer → stores team size and performs code reviews.

Now HR requests that every SeniorDeveloper should be able to access employee information and programming language details without duplicating code.

### Questions:

1. Why is multilevel inheritance suitable here?

Each class builds incrementally on the one before it (Employee → Developer → SeniorDeveloper), and each level genuinely "is-a" refinement of the previous one, so fields/behavior naturally cascade down without duplicating code at each stage.

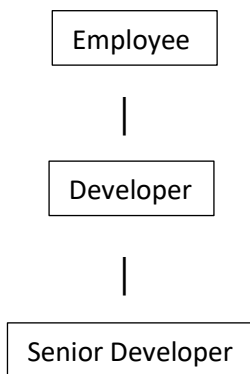
2. Which members are accessible in SeniorDeveloper?

id, name (from Employee), programmingLanguage (from Developer), plus its own teamSize and codeReview() — assuming these members aren't marked private (private members exist in memory but aren't directly accessible by name in the subclass).

3. If Developer overrides a method from Employee, which implementation will SeniorDeveloper inherit?

SeniorDeveloper inherits Developer's overridden version, unless SeniorDeveloper itself overrides it again — Java always resolves to the nearest override up the chain from the actual object's class.

4. Draw the inheritance hierarchy.



### 3. Hierarchical Inheritance – Scenario

#### Scenario:

An online shopping application has a base class Payment.

Three classes inherit from it:

- CreditCardPayment
- UPIPayment
- NetBankingPayment

Each payment type processes transactions differently but shares common validation logic.

#### Questions:

1. Why is hierarchical inheritance appropriate here?

All three payment types share a common concept (Payment) and common behavior (validation), but differ only in one specific step (transaction processing) — a single parent with multiple children models that "one common base, many variants" shape directly.

2. Where should common validation logic be implemented?

In the Payment base class as a concrete method, so all subclasses inherit it automatically instead of re-implementing it.

3. How can each payment class provide its own transaction processing?

By overriding an abstract (or virtual) processTransaction() method declared in Payment.

4. What is the advantage over writing three independent classes?

No duplicated validation code, a single point of maintenance for shared logic, and polymorphic handling — you can treat all payment types uniformly as `Payment` references (e.g., in a list) while still getting the correct behavior for each.

---

#### 4. Compile-Time Polymorphism (Method Overloading) – Scenario

##### Scenario:

A calculator application supports addition of:

- Two integers
- Three integers
- Two decimal numbers

The product owner wants users to call the same method name (`add`) regardless of the input.

##### Questions:

1. Which OOP concept should be used?

Method overloading (compile-time/static polymorphism).

2. How will Java decide which method to invoke?

At compile time, based on the number, type, and order of arguments in the method signature — not the return type.

3. Can methods be overloaded by changing only the return type?

No. Return type alone doesn't make a valid overload; the compiler must be able to distinguish methods by their parameter list, or it's a compile error ("method already defined").

4. What happens if `add(5, 10.0)` is called?

No exact (`int`, `double`) overload exists, so Java applies widening conversion: `5` is promoted to `5.0`, and the `add(double, double)` version is invoked, returning `15.0`.

---

## 5. Runtime Polymorphism (Method Overriding) – Scenario

### Scenario:

A food delivery application has:

Restaurant

|  
-----

|       |       |  
Pizza   Burger   Biryani

Each restaurant calculates delivery time differently.

The application stores all restaurants in:

List<Restaurant> restaurants;

and calls:

restaurant.calculateDeliveryTime();

without checking the actual restaurant type.

### Questions:

1. Which OOP concept is being used?

Runtime polymorphism (dynamic method dispatch) through method overriding.

2. Which version of calculateDeliveryTime() executes?

The version belonging to the object's actual runtime type (Pizza, Burger, or Biryani) — not the declared reference type (Restaurant).

3. When is the method selection decided?

At runtime, using the actual object's type via the JVM's virtual method table — this is what "dynamic dispatch" means, as opposed to overloading's compile-time resolution.

4. What would happen if calculateDeliveryTime() were declared static?

Static methods aren't polymorphic — they're resolved at compile time based on the reference type. Since the list is typed List<Restaurant>, every call would resolve to Restaurant's static method (or cause a compile error if Restaurant has none), and each

subclass's own version would never run through the interface — this is called method hiding, not overriding.

---

## 6. Interface – Scenario

### Scenario:

A smart home application supports different devices:

- Smart Bulb
- Smart Fan
- Smart AC

All devices can be turned on and turned off, but each device has its own implementation.

In the future, third-party manufacturers should also be able to integrate their own devices without modifying existing code.

### Questions:

1. Would you use an interface or inheritance? Why?

Interface. The devices are unrelated in implementation (a bulb and a fan share no common code or state) but agree on a common *contract* (on/off behavior) — and the requirement that third-party manufacturers can add new devices without touching existing code is exactly what interfaces are for, since Java allows implementing an interface from any class hierarchy.

2. Design an appropriate interface.

Switchable with turnOn() and turnOff()

3. Can one device implement multiple interfaces? Give an example.

Yes — Java classes can implement multiple interfaces (unlike single class inheritance). Example: SmartAC implements Switchable, Coolable, combining power control with temperature control.

4. Why is an interface better than creating a common base class in this scenario?

- Java only allows single class inheritance, so a base class would block a device from extending anything else it might need.

- Interfaces impose no shared implementation or state, keeping devices decoupled — a SmartBulb and SmartFan genuinely have nothing in common besides the contract.
  - Third-party manufacturers can implement Switchable in their own class hierarchy without needing to extend a specific base class from your codebase, satisfying the open/extensible design requirement.
-